



**Relatório Final do Sistema de Navegação Autônoma
Hierárquica em CoppeliaSim: Integração de Percepção por
Visão, Planejamento Global A* e Controle Local DWA**

Discente: Adhemar Luiz Cavalcanti Silva Rocha Neto
Docente: Juracy Emanuel Magalhães

Repositório Do DWA: github.com/AdhemarCavalcanti/Rob-tica_2026.1

Repositório Do projeto final: github.com/AdhemarCavalcanti/robotica_projeto_final

1. INTRODUÇÃO

1.1 Contextualização

Orientação de Preenchimento: Introduzir o tema de navegação autônoma de robôs móveis. Citar a importância da simulação em ambientes virtuais (como o CoppeliaSim) e a relevância de estratégias híbridas que unem o planejamento de rotas global (como o algoritmo A*) e desvio de obstáculos local (como o DWA).

1.2 Problema

Orientação de Preenchimento: Definir o desafio técnico que o sistema resolve: como guiar um robô de uma posição inicial até um alvo estipulado de forma segura, contornando tanto barreiras estáticas predefinidas quanto novos obstáculos dinâmicos identificados em tempo real por sensores de proximidade.

1.3 Motivação

Orientação de Preenchimento: Explicar as razões práticas e acadêmicas para o desenvolvimento do projeto. Destacar a eficiência de acoplar algoritmos geométricos/gráficos com controle cinemático em tempo real para evitar colisões e travamentos (*stuck*).

1.4 Objetivo Geral

Orientação de Preenchimento: Declarar o objetivo principal (ex: Desenvolver e implementar um sistema de navegação autônoma e desvio de obstáculos para um robô móvel diferencial utilizando a API remota do CoppeliaSim integrada com algoritmos em Python).

1.5 Objetivos Específicos

Orientação de Preenchimento: Listar em tópicos as metas parciais necessárias para alcançar o objetivo geral:

- Mapear o ambiente do simulador e extrair a matriz de obstáculos.
- Implementar o algoritmo A* para traçar a rota global ideal.
- Configurar o controlador DWA para a tomada de decisão cinemática local.
- Desenvolver lógica de recuperação para situações onde o robô fique preso.

1.6 Escopo do Projeto

Orientação de Preenchimento: Delimitar as fronteiras do projeto (o que está e o que não está incluso). Exemplo: foca-se na simulação cinemática via API

remota de um robô diferencial e sensores de proximidade, não abordando a construção do hardware físico real.

1.7 Organização do Relatório

Orientação de Preenchimento: Descrever brevemente a estrutura do documento, resumindo o que o leitor encontrará em cada um dos capítulos seguintes.

2. FUNDAMENTAÇÃO TEÓRICA

2.1 Robótica Móvel Diferencial

- **Definição:** *(Espaço reservado para a descrição cinemática de robôs móveis com duas rodas independentes).*
- **Funcionamento:** *(Espaço para explicar como a variação de velocidade entre as rodas altera o centro de rotação instantâneo).*
- **Equações:** *(Inserir as matrizes e fórmulas de transição de estado cinemático do robô diferencial).*
- **Referências:** *(Indicar as fontes bibliográficas utilizadas para este conceito).*

2.2 Algoritmo de Busca A* (A-Star)

- **Definição:** *(Apresentação do algoritmo de busca heurística em grafos/malhas).*
- **Funcionamento:** *(Explicação do cálculo de custo total $f(n) = g(n) + h(n)$).*
- **Aplicações:** *(Planejamento de caminhos globais estáticos em matrizes de ocupação).*
- **Vantagens e Limitações:** *(Garantia de otimalidade vs. alto custo computacional em mapas dinâmicos enormes).*
- **Referências:** *(Indicar as fontes bibliográficas).*

2.3 Algoritmo DWA (Dynamic Window Approach)

- **Definição:** *(Abordagem local baseada no espaço de velocidades do robô).*
- **Funcionamento:** *(Amostragem de comandos (v, w) viáveis dentro da janela dinâmica filtrando trajetórias que colidem).*
- **Equações:** *(Inserir a função de custo do DWA que balanceia direção ao alvo, velocidade e distância dos obstáculos).*
- **Referências:** *(Indicar as fontes bibliográficas).*

2.4 Ambientes de Simulação e Integração Computacional

- **Conceito:** *(Revisão teórica sobre o simulador CoppeliaSim e a comunicação cliente-servidor através do protocolo ZMQ Remote API).*

2.5 Tabelas e Figuras Comparativas da Seção

*(Espaço reservado para inclusão de tabelas e diagramas que ilustrem os conceitos teóricos levantados, como o esquema geométrico do DWA ou a expansão de nós do A).**

Tecnologia	Versão	Finalidade	Justificativa
Python	3.x	Linguagem principal do projeto	Alta disponibilidade de bibliotecas científicas e de robótica.
CoppeliaSim	4.x (ou superior)	Ambiente de simulação 3D	Motor de física robusto e suporte a sensores realistas.
ZMQ Remote API	Atual	Protocolo de comunicação síncrona	Permite o controle do simulador via scripts externos de forma ágil.
NumPy	Atual	Processamento matricial e vetorial	Manipulação eficiente dos estados $[x, y, \theta, v, \omega]$ e nuvens de pontos.
Matplotlib	Atual	Renderização de gráficos estáticos	Plotagem prévia do mapa e das rotas geradas pelo planejador A*.
Visual Studio Code / IDE	—	Ambiente de Desenvolvimento	Ferramenta para codificação, depuração e organização do script.

4. ARQUITETURA GERAL DO SISTEMA

4.1 Visão Geral do Sistema

Orientação de Preenchimento: Descrever o ecossistema do projeto em nível macro: o script em Python agindo como o cérebro (controlador) e o CoppeliaSim como o corpo (atuadores e sensores) que responde aos estímulos.

4.2 Diagrama de Arquitetura (Hardware-in-the-loop / Software)

(Inserir aqui o diagrama que ilustra o fluxo de dados entre o Script Principal e as APIs do CoppeliaSim).

4.3 Componentes do Sistema

Orientação de Preenchimento: Detalhar a divisão modular adotada:

1. Módulo de Conectividade (RemoteAPIClient)
2. Módulo de Percepção (mapa_ocupacao e Leitura de Sensores)
3. Módulo de Planejamento Global (AStarPlanner)
4. Módulo de Controle Local e Execução (DWAController / CoppeliaRobotNavigator)

4.4 Fluxo de Funcionamento (Ciclo de Runtime)

Orientação de Preenchimento: Descrever a linha do tempo de execução: Inicialização, Carregamento do Mapa, Planejamento A*, Loop Principal (Leitura de Sensores -> Cálculo DWA -> Atualização de Atuadores) até atingir o Alvo.

4.5 Comunicação entre Módulos

Orientação de Preenchimento: Explicar como as variáveis calculadas pelo planejador são transmitidas para as funções que definem as velocidades angulares das juntas do robô no simulador.

5. MODELAGEM DO SISTEMA

5.1 Diagrama de Classes (UML)

(Inserir aqui o Diagrama de Classes detalhando a estrutura da classe CoppeliaRobotNavigator, seus atributos como state, sensors, dwa, e seus métodos como bootstrap_system(), runtime_loop(), etc.).

5.2 Fluxo de Dados e Sequência de Inicialização

(Inserir aqui o Diagrama de Sequência mapeando as chamadas de funções desde o bloco principal __main__ até a inicialização dos motores e carregamento de dados do Floor).

6. METODOLOGIA DE DESENVOLVIMENTO

6.1 Metodologia Utilizada

Orientação de Preenchimento: Descrever o processo de engenharia adotado para a construção do projeto (por exemplo, desenvolvimento ágil, incremental ou baseado em prototipagem). Explique que o sistema foi desenvolvido em

módulos isolados e validados passo a passo: primeiro estabelecendo a comunicação, depois a extração do mapa, o algoritmo de busca global, o controle dinâmico e, por fim, as rotinas de tratamento de travamento.

6.2 Organização do Projeto e Ferramentas de Gestão

Orientação de Preenchimento: Discorrer sobre como os arquivos de código foram divididos e organizados no diretório de trabalho. Se aplicável, mencione o uso de sistemas de controle de versão (como Git/GitHub) para o rastreamento das alterações do código de navegação e como as sessões de testes no CoppeliaSim foram conduzidas.

7. IMPLEMENTAÇÃO

7.1 Estrutura de Inicialização (bootstrap_system)

Orientação de Preenchimento: Explique o processo de inicialização do robô. Detalhe como a classe localiza os componentes na árvore de objetos da cena do CoppeliaSim (MOTOR_DIREITO, MOTOR_ESQUERDO, SENSOR_MEIO, Alvo), bem como a captura do posicionamento inicial (z) e das orientações de inclinação (*roll* e *pitch*) para calibração do odômetro virtual.

7.2 Módulo de Percepção e Extração de Obstáculos

Orientação de Preenchimento: Detalhar as funções matemáticas criadas para construir o mapa de pontos. Explique a lógica da função `transform_point_3d` (que utiliza a matriz de transformação do objeto para converter coordenadas locais dos sensores em coordenadas globais do mundo) e os métodos de preenchimento de geometria estática (`extract_filled_rectangle` e `generate_bounding_box_points`).

7.3 Módulo de Atuação e Cinemática Diferencial

Orientação de Preenchimento: Explicar o funcionamento físico do método `drive_actuators()`. Detalhar como o script realiza a conversão das velocidades lineares (v) e angulares (w) ditadas pelo algoritmo em velocidades angulares para as juntas das rodas direita (w_{right}) e esquerda (w_{left}), utilizando o raio da roda (0.0375 m) e o espaçamento entre eixos (0.15 m), aplicando também os limites de saturação de ± 20.0 rad/s.

7.4 Lógica de Recuperação para Estado Travado (*Stuck*)

Orientação de Preenchimento: Descrever a lógica preventiva de travamento. Explique o monitoramento feito através da variável `stuck_steps`. Se a posição do robô variar menos do que 0.05 m ao longo de 30 iterações, o sistema assume que o robô colidiu ou ficou preso, ativando a função `calculate_escape_velocity()` por 35 passos para forçar uma marcha à ré e rotação de escape, além de invocar o replanejador do A*.

8. ALGORITMOS UTILIZADOS

8.1 Algoritmo Estrutural: Script Principal (main_robo.py)

Orientações para o Documento: *(Insira aqui o código completo do arquivo main_robo.py que você já possui. Utilize a formatação de código do Word ou bloco de texto com recuo no Google Docs).*

8.2 Controlador Local: Janela Dinâmica (dwa_navigation.py)

Orientações para o Documento: *(Espaço reservado para você colar o código fonte do arquivo dwa_navigation.py. Adicione abaixo uma breve explicação de como o código lida com a amostragem de velocidades).*

8.3 Mapeamento de Cena: Matriz de Ocupação (mapa_ocupacao.py)

Orientações para o Documento: *(Espaço reservado para você colar o código fonte do arquivo mapa_ocupacao.py. Adicione abaixo uma breve explicação sobre como o código varre e filtra os objetos do cenário do CoppeliaSim).*

9. BANCO DE DADOS (NÃO APLICÁVEL)

Nota de Configuração do Projeto: Esta seção foi mantida para fins de padronização de relatórios de engenharia, porém é classificada como **Não Aplicável (N/A)** para este escopo. Os dados geométricos de obstáculos e estados dinâmicos do robô $[x, y, \theta, v, \omega]$ são processados e estruturados puramente em memória volátil de tempo de execução (NumPy arrays), descartando a necessidade de persistência em sistemas gerenciadores de banco de dados (SGBD).

10. INTERFACE DO SISTEMA

10.1 Interface de Visualização Prévia (Matplotlib)

- **Objetivo:** Apresentar ao operador a malha geométrica e as rotas calculadas antes de liberar a simulação física.
- **Funcionalidades:** Plotagem em tempo real de duas janelas (*subplots*): uma contendo a "Rota Completa" (caminho contínuo do A*) e outra destacando os "Pontos-Chave" (pontos de inflexão e mudança de direção que guiarão o DWA).
- **Espaço para Captura de Tela:** *(Inserir o print da tela de gráficos gerada pelo script Python antes do início da simulação).*

10.2 Ambiente de Simulação Ativa (CoppeliaSim)

- **Objetivo:** Monitorar o comportamento dinâmico e o deslocamento do robô pelo mapa 3D.
- **Funcionalidades:** Visualização dos sensores atuando, renderização física das colisões e do atingimento do objeto /Alvo.
- **Espaço para Captura de Tela:** *(Inserir um ou mais prints do CoppeliaSim mostrando o robô navegando e desviando de obstáculos).*

11. INTEGRAÇÃO DOS COMPONENTES

Orientação de Preenchimento: Descrever detalhadamente como o ecossistema de software se conecta. Explique o ciclo fechado: o simulador fornece as posições e leituras de sensores via ZMQ Remote API; o módulo mapa_ocupacao processa essas coordenadas; o AStarPlanner gera a rota global; o DWAController traduz essa rota em comandos cinemáticos locais; e a função drive_actuators envia os comandos de velocidade de volta para os motores do CoppeliaSim, reiniciando o ciclo a cada step().

12. METODOLOGIA DE TESTES

12.1 Estratégia de Homologação e Cenários de Validação

Orientação de Preenchimento: Descrever detalhadamente a estratégia adotada para testar o sistema de navegação e integração. Explique que as validações foram divididas em ensaios sequenciais, nos quais cada funcionalidade crítica do script foi estressada isoladamente antes do teste integrado em ambiente dinâmico.

12.2 Caso de Teste 01 — Validação de Conectividade e Inicialização

- **Objetivo do Teste:** Verificar o comportamento do script ao tentar abrir a comunicação síncrona via ZMQ Remote API.
- **Condição de Entrada:** Execução do arquivo principal (main_robo.py) com o software CoppeliaSim fechado ou sem nenhuma cena ativa.
- **Resultado Esperado:** O programa deve interceptar a ausência do simulador, emitir uma exceção ou mensagem de falha limpa no terminal e encerrar a execução sem travar o sistema operacional.
- **Resultado Obtido:** *(Preencher com os logs ou comportamentos observados na sua execução).*
- **Status:** *(Aprovado / Reprovado)*

12.3 Caso de Teste 02 — Extração e Filtragem da Nuvem de Obstáculos

- **Objetivo do Teste:** Garantir que o método load_static_map_data consegue ler a árvore de objetos e gerar a matriz geométrica de colisões.
- **Condição de Entrada:** Execução do bootstrap do sistema em uma cena contendo o plano /Floor e múltiplos obstáculos geométricos.
- **Resultado Esperado:** O script deve listar os handles, aplicar as exclusões de proximidade do robô/alvo e salvar um array NumPy estruturado contendo as coordenadas (x, y) exclusivas de cada barreira.
- **Resultado Obtido:** *(Preencher descrevendo se o fallback de Bounding Box precisou ser ativado ou se a visão construiu o mapa).*
- **Status:** *(Aprovado / Reprovado)*

12.4 Caso de Teste 03 — Convergência do Planejador Global A*

- **Objetivo do Teste:** Validar a capacidade do algoritmo de busca em gerar uma linha de waypoints viável em cenários complexos.
- **Condição de Entrada:** Robô posicionado em uma extremidade do mapa e o objeto /Alvo fixado na extremidade oposta, com barreiras entre eles.
- **Resultado Esperado:** O algoritmo deve iterar sobre diferentes raios de segurança do robô ($rr = 0.22, 0.18$ ou 0.14) e gerar as listas de rota completa (`global_path`) e pontos de inflexão (`key_points`).
- **Resultado Obtido:** *(Preencher indicando com qual raio de segurança o algoritmo A obteve sucesso na sua simulação).**
- **Status:** *(Aprovado / Reprovado)*

12.5 Caso de Teste 04 — Desvio Local e Rastreamento Dinâmico (DWA)

- **Objetivo do Teste:** Testar a capacidade do controlador Dynamic Window Approach em desviar de novos obstáculos usando os sensores de proximidade de forma ativa.
- **Condição de Entrada:** Inserção ou aproximação de um obstáculo não mapeado na trajetória calculada pelo A* enquanto o robô se desloca.
- **Resultado Esperado:** Os sensores `/SENSOR_MEIO`, `/SENSOR_DIREITO`, etc., devem registrar a proximidade, agregá-la ao mapa dinâmico local e o DWA deve recalculer a janela de velocidades (v, ω) para contornar o objeto sem colidir.
- **Resultado Obtido:** *(Preencher descrevendo o comportamento dinâmico do robô ao passar perto das quinas ou novas barreiras).*
- **Status:** *(Aprovado / Reprovado)*

12.6 Caso de Teste 05 — Ativação e Resolução do Módulo de Escape (*Stuck*)

- **Objetivo do Teste:** Validar a robustez da rotina de recuperação em cenários de enclausuramento ou obstrução total.
- **Condição de Entrada:** Bloquear completamente a passagem do robô, forçando-o a colidir ou ficar retido sem conseguir avançar em direção ao waypoint.
- **Resultado Esperado:** O script deve acumular 30 iterações com variação de posição inferior a 0.05 m, incrementar `stuck_steps`, disparar o aviso de travamento no console, acionar a função `calculate_escape_velocity()` (forçando marcha à ré) e reiniciar o planejador A* global.
- **Resultado Obtido:** *(Preencher detalhando se o robô conseguiu se desvencilhar do bloqueio simulado).*
- **Status:** *(Aprovado / Reprovado)*

13. RESULTADOS OBTIDOS

Orientação de Preenchimento: Exibir os dados empíricos coletados durante as sessões de simulação bem-sucedidas. Apresente em formato de texto descritivo métricas quantitativas, como: o número total de passos de simulação (`step_count`) demandados até o encerramento do trajeto, a distância linear final restante entre o centro do robô e o alvo, e o comportamento cinemático ao exibir a mensagem conclusiva "ALVO ATINGIDO!". Reserve também

subparágrafos para descrever o perfil de velocidade angular aplicado às juntas dos motores durante as curvas fechadas.

14. DISCUSSÃO

Orientação de Preenchimento: Realizar uma análise crítica e qualitativa dos testes documentados. Discuta os seguintes fenômenos técnicos: as oscilações ou pequenos tremores na orientação angular gerados devido à amostragem discreta da janela dinâmica do DWA, a influência do parâmetro `heading_offset` calculado na calibração inicial e as situações limítrofes onde o robô quase excedeu o raio de colisão (`collision_radius`) em quinas muito agudas antes do acionamento da rotina de escape.

15. RELAÇÃO ENTRE TEORIA E IMPLEMENTAÇÃO

Orientação de Preenchimento: Explicar cientificamente como os fundamentos de engenharia e computação deram suporte ao código escrito. Demonstre a correspondência direta entre:

1. As equações de transformações homogêneas e matrizes de rotação espacial com o método trigonométrico implementado em `transform_point_3d`.
2. Os modelos cinemáticos diretos de robôs diferenciais com a distribuição de rotações feita em `drive_actuators` (equacionando raio da roda de 0.0375 m e bitola de 0.15 m).
3. A teoria de espaços de busca e grafos discretos com as tentativas de iteração por raios decrescentes do planejador A*.

16. CONCLUSÃO

16.1 Resumo do Trabalho

Orientação de Preenchimento: Elaborar uma síntese do projeto técnico desenvolvido, ressaltando o cumprimento da meta de criar um controlador integrado capaz de gerenciar percepção, planejamento de caminhos e atuação física simulada em tempo de execução síncrono.

16.2 Objetivos Alcançados

Orientação de Preenchimento: Confirmar explicitamente a validação de cada um dos objetivos gerais e específicos delimitados no Capítulo 1, amparando-se nos resultados bem-sucedidos obtidos nos testes de rodagem e na estabilidade da conexão baseada na biblioteca `RemoteAPIClient`.

16.3 Limitações e Trabalhos Futuros

Orientação de Preenchimento: Evidenciar as restrições operacionais do sistema atual (como a suscetibilidade a travamentos caso os parâmetros de aceleração do DWA estivessem descalibrados ou a dependência de um mapa

estático perfeitamente lido no início). Recomende linhas de pesquisa futuras para a continuidade do projeto, tais como a adoção de filtros de Kalman para localização assistida por odometria ruidosa, implementação de varredura ativa via sensores LiDAR e fusão de dados em tempo real para algoritmos de SLAM.

17. EVIDÊNCIAS DO PROJETO

Orientação de Preenchimento: Dedicar este espaço a um parágrafo que referencie todas as capturas de tela e materiais de amostragem gerados durante o desenvolvimento. Indique de forma textual o nome de cada figura anexada ao corpo do texto (como a visualização estática do Matplotlib e a janela ativa em 3D do CoppeliaSim), descrevendo o que cada imagem comprova tecnicamente em termos de engenharia.